

Kapitel 16

Ein-/Ausgabe

16.1 Verarbeitung von Dateien

Längst nicht in jedem Falle kommt man mit Standardein- und -ausgabe aus, auch nicht unter Berücksichtigung der Möglichkeit, beide umzuleiten.

Ein Beispiel: Ein Programm soll zwei in sich sortierte Dateien „mischen“, d.h. es soll die beiden Dateien parallel lesen und ihre Inhalte sortiert in eine dritte Datei schreiben. Dieses lässt sich prinzipiell mit Umleitung der Standardeingabe *nicht* realisieren, weil Umleitung immer voraussetzt, dass man die Eingabe aus einer einzigen Datei liest.

Abhilfe schafft die Möglichkeit, direkt auf Dateien zuzugreifen. Bevor man aber direkt auf eine Datei zugreifen kann, muss man eine Vorarbeit vornehmen: Dateien werden ja vom Betriebssystem verwaltet; zu ihrer Identifikation dienen Namen. Diese Namen kann man in einem C-Programm für die eigentlichen Zugriffe nicht verwenden, sondern nur im Zuge der Vorarbeit, nämlich durch Zuordnung des (externen) Namens der Datei zu einer (programminternen) *Dateivariablen*.

Für diese Zuordnung stellt `stdio.h` die Funktion

```
FILE *fopen (const char *dateiname ,
             const char *zugriff);
```

zur Verfügung:

- Der erste Parameter ist der Zeiger auf den String, der den externen Namen der Datei enthält.
- Der zweite Parameter ist der Zeiger auf einen String, der die gewünschte Art des Zugriffs beschreibt. Eine Übersicht über die Optionen ist in Tabelle 16.1 aufgelistet.
- Der Funktionswert ist der Zeiger auf eine Dateivariablen, falls die Zuordnung vorgenommen werden konnte, oder der Nullzeiger. Der Typ `FILE` ist ebenfalls in `stdio.h` deklariert.

Die Zeiger, die `fopen` liefert, sind ausschließlich zur Weitergabe an die anderen Funktionen bestimmt, die Dateien bearbeiten. Der Standard legt auch keinerlei Einzelheiten des Aussehens des Typs `FILE` fest; in der Regel wird es sich um einen Strukturtyp handeln, der aber je nach Implementation völlig unterschiedlich aussehen kann.

Das Pendant zu `fopen` ist die Funktion

```
int fclose (FILE *datei);
```

Option	Beschreibung
r	Datei lesen; die Datei muss bereits existieren
w	Datei schreiben; falls die Datei existiert, wird ihr Inhalt überschrieben
a	Datei fortsetzen; erstellt Datei, falls sie nicht existiert
r+	Datei lesen und schreiben; die Datei muss bereits existieren
w+	Datei lesen und schreiben; falls die Datei existiert, wird ihr Inhalt überschrieben
a+	Datei lesen und fortsetzen; erstellt Datei, falls sie nicht existiert

Tabelle 16.1: Optionen zum Öffnen von Dateien mit **fopen**

Sie löst die Zuordnung für die übergebene Dateivariablen und meldet mit dem Funktionswert Null bzw. EOF ihren Erfolg bzw. Misserfolg.

Lesen und schreiben können wir jetzt nicht mehr mit **scanf**, **getchar**, **printf** und **putchar**, weil wir natürlich die betroffene Datei zusätzlich nennen müssen. Die entsprechenden Funktionen

```
int fscanf(FILE *datei, const char *format, ...);
int fgetc(FILE *datei);
int fprintf(FILE *datei, const char *format, ...);
int fputc(int c, FILE *datei);
```

arbeiten aber letztlich genauso wie die bekannten Funktionen, nur dass sie eben gerade auf die angegebene Datei anstelle der Standardein- bzw. -ausgabe zugreifen.

Als Beispiel wird die Konkatenation von zwei Dateien realisiert:

```
#include <stdio.h>

/** Prototypen *****/
static FILE *zugeordnete_datei(const char *frage,
                                const char *zugriff);
static void datei_kopieren(FILE *ziel, FILE *quelle);

/*= Hauptprogramm =====*/
int main(void) {
    FILE *quelle, *ziel;

    ziel = zugeordnete_datei("Zieldatei: ", "w");
    if (ziel == NULL) {
        printf("Zieldatei nicht verfuegbar -- stop\n");
        return 1;
    }

    quelle = zugeordnete_datei("1. Quelldatei: ", "r");
    if (quelle == NULL) {
        printf("1. Quelldatei nicht verfuegbar -- stop\n");
        return 2;
    }
    datei_kopieren(ziel, quelle);
    fclose(quelle);

    quelle = zugeordnete_datei("2. Quelldatei: ", "r");
```

```

    if (quelle == NULL) {
        printf("2. Quelldatei nicht verfuegbar -- stop\n");
        return 3;
    }
    datei_kopieren(ziel, quelle);
    fclose(quelle);

    fclose(ziel);
    return 0;
}

/*= Zuordnung einer Datei (mit Abfrage ihres Namens) =====*/
static FILE *zugeordnete_datei(const char *frage,
                                const char *zugriff) {
    char name[FILENAME_MAX];
    int i;
    FILE *datei;

    printf("%s", frage);
    i = 0;
    while ((i < FILENAME_MAX) && ((name[i] = getchar()) != '\n'))
        i++;
    if (i >= FILENAME_MAX)
        i--;
    if (name[i] != '\n')
        while (getchar() != '\n')
            ;
    name[i] = '\0';

    datei = fopen(name, zugriff);
    return datei;
}

/*= Kopieren einer Datei (zeichenweise) =====*/
static void datei_kopieren(FILE *ziel, FILE *quelle) {
    int c;
    while ((c = fgetc(quelle)) != EOF)
        fputc (c, ziel);
}

```

Quelltext 16.1: Dateiverarbeitung: Konkatenation

16.2 Formatierte und binäre Ein-/Ausgabe

Wie schon erwähnt, muss man die Ein-/Ausgabe, die wir bislang verwendet haben, genauer als „formatierte Ein-/Ausgabe“ bezeichnen. Im Zuge der Übertragung erfolgt in der Regel eine Umwandlung der Darstellung:

- Bei der Eingabe müssen die Zeichen *interpretiert* werden, um herauszufinden, wo ein Eingabewert anfängt und wo er aufhört. Bei numerischen Werten muss dann die Zeichenfolge in die interne, binäre Darstellung des Wertes umgewandelt werden.
- Bei der Ausgabe numerischer Werte muss die interne, binäre Darstellung in eine Zeichenfolge umgewandelt werden.

Das Pendant zur formatierten Ein-/Ausgabe ist die *binäre (unformatierte)* Ein-/Ausgabe: Bei ihr werden die Bitfolgen der Werte im Zuge der Übertragung *nicht* verändert.

Binäre Ein-/Ausgabe hat einen wesentlichen Vorteil: Sie ist in der Regel *sehr viel* schneller als formatierte Ein-/Ausgabe. Das ist auch ohne weiteres einsichtig: Die Umwandlungen zwischen interner Darstellung und Zeichendarstellung erfordern eine Vielzahl von arithmetischen Operationen; wir haben das in einem Beispiel für eine ganze Zahl bereits einmal gesehen. Spart man diese Operationen ein, *muss* das die Übertragung beschleunigen.

Andererseits lässt sich binäre Ein-/Ausgabe aus zwei Gründen nur eingeschränkt einsetzen:

- Nicht jedes Ein-/Ausgabegerät erlaubt binäre Ein-/Ausgabe! Eine Tastatur kann zum Beispiel nur Zeichen liefern; ein Bildschirm oder Drucker kann mit binären Darstellungen nichts anfangen. Bei all diesen Geräten ist man also auf formatierte Ein-/Ausgabe angewiesen.
- Binärdateien sind in der Regel nicht portabel: Ein `int`-Wert zum Beispiel benötigt in der Datei wie im Speicher des Rechners 2 oder 4 Bytes (oder was auch immer); auf einem Rechner, der `int`-Werte anders darstellt, kann die Binärdatei ohne weiteres also nicht gelesen werden. Formatierte Dateien sind zwar auch nicht ohne weiteres portabel, weil die Zeichencodes auf verschiedenen Rechnern verschieden sein können. Die Umsetzung von Zeichencodes ist jedoch extrem simpel, verglichen mit der Umsetzung der internen Darstellungen numerischer Werte.

Anders formuliert: Bei formatierten Dateien hat man immer eine implizite Interpretationsvorschrift, nämlich Interpretation als Zeichen; bei Binärdateien fehlt diese Vorschrift.

Unter UNIX ist es deshalb üblich, weitestgehend formatierte Ein-/Ausgabe und entsprechend formatierte Dateien zu verwenden. Auf die beiden Funktionen `fread` und `fwrite`, mit denen binäre Ein-/Ausgabe möglich ist, wird deshalb auch nicht näher eingegangen.

Die wesentlichsten Funktionen zur formatierten Ein-/Ausgabe kennen wir bereits seit längerem; was nachzutragen bleibt, sind die vielfältigen Möglichkeiten, Einfluss auf die Form der Umwandlung zu nehmen.

16.3 Ausgabeformate

Die Formatbeschreiber für die Ausgabe haben die allgemeine Form

```
%<modus><laenge><.stellen><typ>kennung
```

Wir sehen: Außer dem Prozentzeichen am Anfang und dem Kennbuchstaben *kennung* am Ende sind alle Einträge optional.

Der Formatierungsstring und die Liste der Ausgabewerte werden linear abgearbeitet. Die Abarbeitung beginnt mit der Interpretation des Formatierungsstring. Was weiter passiert, hängt von den dabei gefundenen Zeichen ab:

- Wenn im Formatierungsstring ein Prozentzeichen gefunden wird, wird zunächst der vollständige Formatbeschreiber interpretiert. Dann wird der nächste Ausgabewert ihm entsprechend in eine Zeichenfolge umgewandelt und in die Ausgabedatei übertragen.
- Wenn im Formatierungsstring ein anderes Zeichen gefunden wird, wird es ohne weiteres in die Ausgabedatei übertragen.

Die Kennungen für die verschiedenen Datentypen bzw. ihre externen Darstellungen, die weitgehend am Anfang des Kurses bereits aufgezählt wurden, lassen sich in vier Gruppen unterteilen:

1. Ausgabe ganzer Zahlen

Der Ausgabewert muss den Typ `int` bzw. `unsigned int` besitzen. Die zu erzeugende externe Darstellung wird durch die folgenden Kennbuchstaben beschrieben:

- `i` der Wert wird als `signed int` betrachtet, die Darstellung erfolgt dezimal, ggf. mit Vorzeichen
- `d` wie `i`
- `u` der Wert wird als `unsigned int` betrachtet, die Darstellung erfolgt dezimal
- `o` der Wert wird als `unsigned int` betrachtet, die Darstellung erfolgt oktal
- `x` der Wert wird als `unsigned int` betrachtet, die Darstellung erfolgt hexadezimal unter Verwendung von Kleinbuchstaben
- `X` wie `x`, jedoch mit Großbuchstaben

2. Ausgabe von Gleitkommazahlen

Der Ausgabewert wird als `double` erwartet. Die zu erzeugende externe Darstellung wird durch die folgenden Kennbuchstaben beschrieben:

- `f` Darstellung ohne Exponententeil
- `e` Darstellung mit Exponententeil, der Exponenten-Kennbuchstabe ist `e`; die Darstellung wird so normiert, dass links vom Dezimalpunkt genau eine Ziffer steht, die nur dann Null ist, wenn der Wert insgesamt Null ist
- `E` wie `e`, jedoch mit Exponenten-Kennbuchstabe `E`
- `g` wie `f` oder `e`, je nach Größenordnung des Ausgabewertes; Nullen am Ende der Ziffernfolge werden unterdrückt; der Dezimalpunkt wird nur geschrieben, wenn ihm eine Ziffer folgt
- `G` wie `g`, jedoch wird ggf. `E` anstelle von `e` verwendet

Die letzte Ziffer wird jeweils gerundet.

3. Ausgabe von Zeichen und Strings

- `s` der Ausgabeparameter wird als Zeiger auf einen String betrachtet, der Wert des String geschrieben
- `c` der `int`-Wert wird in `unsigned char` umgewandelt geschrieben

4. Kennungen für verschiedene Zwecke

- `p` der Wert wird als Zeiger betrachtet und in implementations-spezifischer Darstellung geschrieben
- `n` es wird keine Ausgabe erzeugt, sondern die Anzahl der bislang übertragenen Zeichen in den nächsten Parameter der Ausgabeliste geschrieben; dieser Parameter muss entsprechend den Typ `int *` besitzen
- `%` ein Prozentzeichen wird geschrieben (dem Formatbeschreiber wird kein Ausgabewert zugeordnet)

Wenn in einem Formatbeschreiber nur die Kennung *kennung* angegeben ist, werden Werte mit bestimmten Typen erwartet, erfolgt die Darstellung in einem Standardformat. Zum Beispiel wird bei Verwendung des Formatbeschreibers `u` ein `unsigned int`-Wert erwartet. Auch werden bei der Umwandlung nur die signifikanten Ziffern des Wertes erzeugt. Abweichende Werte und Darstellungen lassen sich mit den optionalen Einträgen der Formatbeschreiber anzeigen bzw. erzeugen.

Der Eintrag *laenge* legt die Mindestzahl der zu erzeugenden Zeichen fest, die ggf. durch voran- oder nachgestellte Leerzeichen zu erreichen ist. Falls bei der Umwandlung des Wertes mehr Zeichen entstehen, wird die Angabe ignoriert. *laenge* kann eine dezimale Konstante oder ein Stern sein; wenn ein Stern angegeben ist, wird der nächste Wert der Ausgabeliste als Wert von *laenge* genommen und nicht geschrieben. Dieser Wert *muss* den Typ `int` besitzen.

Negative Längen sind nicht möglich (abgesehen davon, dass sie auch nicht sinnvoll sind): Ein Minuszeichen wird als Modifikator betrachtet und bewirkt linksbündige Ausgabe anstelle der sonst rechtsbündigen, so dass für *laenge* nur die angegebene Ziffernfolge übrig bleibt.

Einige Beispiele: Seien *x* und *y* durch

```
int x = 743, i = 4;
float y = 123.4567;
```

definiert. Dann erhalten wir

<code>printf("%d", x);</code>	→ 743
<code>printf("%2d", x);</code>	→ 743
<code>printf("%6d", x);</code>	→ <code> 743</code>
<code>printf("%-6d", x);</code>	→ <code>743 </code>
<code>printf("%*d", i, x);</code>	→ <code> 743</code>
<code>printf("%f", y);</code>	→ 123.456703
<code>printf("%4f", y);</code>	→ 123.456703
<code>printf("%s", "Hallo");</code>	→ Hallo
<code>printf("Hallo");</code>	→ Hallo
<code>printf("%s", "Hallo%Hallo");</code>	→ Hallo%Hallo
<code>printf("Hallo%Hallo");</code>	→ <i>Fehler!</i>
<code>printf("%10s", "Hallo");</code>	→ <code> Hallo</code>

Wie *stellen* interpretiert wird, hängt in erster Linie von *kennung* ab. Formal kann *stellen* wie *laenge* eine dezimale Konstante oder ein Stern sein; wenn ein Stern angegeben ist, wird der nächste Wert der Ausgabeliste als Wert von *stellen* genommen und nicht geschrieben. Dieser Wert *muss* den Typ `int` besitzen:

- Bei ganzzahligen Werten ist *stellen* die Mindestzahl der zu schreibenden Ziffern, auch wenn dadurch führende Nullen erscheinen.
- Bei den Kennungen `f`, `e` und `E` ist *stellen* die Anzahl der Stellen hinter dem Dezimalpunkt; wird die Anzahl der Stellen hinter dem Dezimalpunkt auf Null gesetzt, entfällt auch der Dezimalpunkt selbst.
- Bei den Kennungen `g` und `G` wird genauso verfahren; nur wird ggf. bei *stellen* der Wert 0 durch 1 ersetzt.
- Bei Strings bewirkt *stellen*, dass höchstens so viele Zeichen des String übertragen werden.

Seien *x* und *y* wie oben definiert. Dann gilt

<code>printf("%6.4d", x);</code>	→ <code> 0743</code>
<code>printf("%-6.4d", x);</code>	→ <code>0743 </code>
<code>printf("%7.2f", y);</code>	→ <code> 123.46</code>

```
printf("%7.3s", "Hallo");      →  ␣␣␣Hal
printf("%7.6s", "Hallo");      →  ␣Hallo␣
```

Dass man Ausgabewerte mit `long`-Typen und dem Typ `long double` durch Angabe des Kennbuchstabens `l` bzw. `L` für *typ* besonders kennzeichnen muss, haben wir bereits gesehen.

Ebenfalls erwähnt wurde bereits das Zeichen `-` als *modus*. Ein weiteres Zeichen, das hier zugelassen ist, ist `+`. Es bewirkt für vorzeichenbehaftete Zahlen, dass auch positive Vorzeichen geschrieben werden.

Für weitere Einzelheiten zu den Ausgabeformaten sei auf die Literatur verwiesen.

16.4 Eingabeformate

Die Formatbeschreiber für die Eingabe haben die Form

```
%<*><laenge><typ>kennung
```

Dabei sind die in spitzen Klammern stehenden Einträge erneut optional.

Der Formatierungsstring, die Zeichen aus der Eingabedatei und die Liste der Eingabevariablen werden linear abgearbeitet. Die Abarbeitung beginnt mit der Interpretation des Formatierungsstring. Was weiter passiert, hängt von den dabei gefundenen Zeichen ab. Zunächst soll der einfachste Fall unterstellt werden, dass nämlich die Formatbeschreiber nur aus dem einleitenden Prozentzeichen und der Kennung *kennung* bestehen, dass keine optionalen Einträge enthalten sind:

- Wenn im Formatierungsstring ein „white space“ gefunden wird, werden so lange Zeichen aus der Eingabedatei geholt, bis das nächste Zeichen *kein* „white space“ ist. Die übertragenen „white spaces“ werden ignoriert.
- Wenn im Formatierungsstring ein Zeichen gefunden wird, das kein Prozentzeichen und kein „white space“ ist, wird es mit dem nächsten Zeichen der Eingabedatei verglichen. Wenn Übereinstimmung besteht, werden beide Zeichen ignoriert. Sonst werden so lange Zeichen in der Eingabedatei übersprungen, bis ein „white space“ gefunden wird, dann die Routine mit Fehlerstatus abgebrochen.
- Wenn im Formatierungsstring ein Prozentzeichen gefunden wird, wird zunächst der vollständige Formatbeschreiber interpretiert. Dann werden so lange Zeichen aus der Eingabedatei geholt, wie diese zur Kennung *kennung* des Formatbeschreibers „passen“. Anschließend wird diese Zeichenfolge in die entsprechende interne Darstellung umgewandelt und das Resultat in der nächsten Eingabevariablen gespeichert.

Die Kennungen für die verschiedenen Datentypen bzw. ihre externen Darstellungen lassen sich wieder in vier Gruppen unterteilen. Wir kennen sie weitgehend bereits:

1. Eingabe ganzer Zahlen

Die Eingabevariable muss den Typ `int` bzw. `unsigned int` besitzen. Die erwartete externe Darstellung wird durch die folgenden Kennbuchstaben beschrieben:

- i ganze Zahl mit oder ohne Vorzeichen in C-Schreibweise, d.h. mit `0x` oder `0X` beginnende Zahlen werden als hexadezimal, andere mit `0` beginnende Zahlen als oktal dargestellt betrachtet; Zahlen, die nicht mit Null beginnen, werden als dezimal dargestellt betrachtet
- d ganze Zahl mit oder ohne Vorzeichen in dezimaler Darstellung
- u ganze Zahl ohne Vorzeichen in dezimaler Darstellung

- o ganze Zahl mit oder ohne Vorzeichen in oktaler Darstellung
- x ganze Zahl mit oder ohne Vorzeichen in hexadezimaler Darstellung, jedoch ohne einleitendes 0x bzw. 0X
- X wie x

2. Eingabe von Gleitkommazahlen

Die Eingabevariable muss den Typ `float` besitzen. Die verfügbaren Kennungen sind die Buchstaben `e`, `E`, `f`, `g` und `G`. Sie erwarten sämtlich eine beliebige zulässige Darstellung einer Gleitkommazahl, also mit oder ohne Vorzeichen, mit oder ohne Dezimalpunkt, mit oder ohne Exponententeil.

3. Eingabe von Zeichen und Strings

Die Eingabevariable muss den Typ `char` besitzen; in der Regel wird sie die erste Komponente eines Feldes sein müssen, das hinreichend groß ist, alle übertragenen Zeichen aufzunehmen. Die erwartete externe Darstellung wird durch die folgenden Kennungen beschrieben:

- s beliebige Zeichenfolge, beendet durch ein „white space“; an die Zeichenfolge wird automatisch ein Null-Zeichen angehängt
- [wie s, jedoch kann explizit angegeben werden, bei welchen Zeichen die Übertragung stoppen soll (vgl. unten)
- c einzelnes Zeichen, das auch ein „white space“ sein kann

4. Kennungen für verschiedene Zwecke

- p Erwartet wird ein Zeigerwert in implementations-spezifischer Darstellung; die Eingabevariable muss einen Zeigertyp besitzen. Die Kennung ist, wenn überhaupt, nur dann sinnvoll, wenn die zu lesende Eingabe unter derselben Implementation geschrieben wurde.
- n Es werden keine Zeichen aus der Eingabedatei geholt, sondern die Anzahl der bisher umgewandelten Werte in die nächste Eingabevariable übertragen. Diese Eingabevariable muss den Typ `int` besitzen.
- % Kennung für ein Prozentzeichen, das als Eingabezeichen erwartet wird.

Die Kennbuchstaben für numerische Werte müssen durch die Typkennung `h`, `l` oder `L` modifiziert werden, wenn die Eingabevariable *nicht* den Typ `int` bzw. `float` besitzt. Durch *laenge* kann man angeben, wie viele Zeichen maximal für den Eingabewert interpretiert werden sollen.

Als Beispiel betrachten wir die Anweisungsfolge

```
int i, anzahl;
char c;
float f;
double d;
...
anzahl = scanf("%5d %c %f %lf", &i, &c, &f, &d);
```

Dann bewirkt die Eingabezeile

```
12345Y3.14 2.12
```

die Zuweisungen


```
i = 12345;
c = 'Y';
f = 3.14;
d = 2.12;
anzahl = 4;
```

Durch die Eingabezeilen

```
12345 Y3.14 2.12
12345 Y 3.14 2.12
```

ändert sich am Resultat nichts (wegen des Leerzeichens in `□%c !!`). Schreiben wir dagegen

```
123456Y3.14 2.12
```

so werden die Zuweisungen

```
i = 12345;
c = '6';
anzahl = 2;
```

ausgeführt, während die Zuweisungen für `f` und `d` unterbleiben.

Beginnt ein Formatbeschreiber mit einem Stern, so wird er zwar bei der Interpretation der Eingabe „normal“ abgearbeitet – der gelesene Wert wird jedoch nicht in einer Eingabevariablen gespeichert. Auch dafür ein Beispiel:

```
int i, o, anzahl;
char c;
float f;
...
anzahl = scanf("%3d zahl %4o %*d %c %f", &i, &o, &c, &f);
```

Mit den Eingabezeilen

```
473 zahl11563 567 R2.33E+2
473 zahl11568 567 R2.33E+2
473z ah 11563 567 R2.33E+2
```

erhalten wir jetzt

```
i = 473;           i = 473;           i = 473;
o = 883;           o = 110;
c = 'R';           c = '5';
f = 233.0;         f = 67;
anzahl = 4;        anzahl = 4;        anzahl = 1;
```

Eine interessante Möglichkeit ist, für Strings die zulässigen oder nicht zulässigen Zeichen explizit anzugeben:

```
[zeichenfolge]
[~zeichenfolge]
```

Wir haben in einem Beispiel einmal mit `getchar` in einer Schleife dafür gesorgt, dass aus jeder Eingabe nur die Zahl am Anfang interpretiert und der Rest der Eingabezeile ignoriert wurde:

```
scanf("%d", &wert);           /* eine Zahl lesen */
while (getchar() != '\n')     /* Rest ignorieren */
    ;
```

Jetzt können wir auch kürzer

```
scanf("%d%*[^\\n]", &wert); getchar ();
```

schreiben. Achtung: Das Format `%d%*[^\\n]%*c` bei gleichzeitigem Verzicht auf den Aufruf von `getchar` ist *nicht* möglich. Wird für den Formatbeschreiber `%*[^\\n]` kein zulässiges Zeichen gefunden, so wird die Funktion mit einem Fehler beendet, der Formatbeschreiber `%*c` also gar nicht mehr bearbeitet. Immer dann, wenn das Zeilenende-Zeichen der Zahl unmittelbar folgt, würde es also im Puffer stehenbleiben.

16.5 Funktionen zur Ein-/Ausgabe

Das Funktionenpaar

```
int scanf(const char *format, ...);
int printf(const char *format, ...);
```

kennen wir bereits seit Beginn des Kurses: Es erlaubt den Zugriff auf Standardein- und -ausgabe. Den Zugriff auf beliebige Dateien erlaubt das Funktionenpaar

```
int fscanf(FILE *datei, const char *format, ...);
int fprintf(FILE *datei, const char *format, ...);
```

das am Anfang dieses Abschnitts eingeführt wurde. `scanf` und `printf` dienen ausschließlich der Bequemlichkeit des Programmierers: Der Standard schreibt vor, dass für die Standardein- und -ausgabe die Namen `stdin` bzw. `stdout` vordefiniert sind, so dass man grundsätzlich auch

```
fscanf(stdin, ...);
fprintf(stdout, ...)
```

schreiben könnte. (Es gibt übrigens mit `stderr` noch eine dritte „Standarddatei“ für die Ausgabe von Fehlermeldungen. Sie wird bei interaktiven Programmen in der Regel auch dem Bildschirm zugeordnet.)

Es gibt noch ein drittes, in manchen Fällen interessantes Funktionenpaar, nämlich

```
int sscanf(const char *s, const char *format, ...);
int sprintf(char *s, const char *format, ...);
```

Auch diese Funktionen führen Umwandlungen zwischen interner und externer Darstellung durch, nur greifen sie nicht auf eine Datei zu, sondern statt dessen auf den String, der als erster Parameter übergeben wird. Letztlich werden damit dem Programmierer nur die Umwandlungsroutinen für seine Zwecke zur Verfügung gestellt, die hinter den Formatbeschreibern stehen.

16.6 Vorausschau beim Lesen

Eine weitere Funktion möchte zum Abschluss dieses Abschnitts noch ansprechen, nämlich

```
int ungetc(int c, FILE *datei);
```

Dahinter steht: Bei der Interpretation von Eingabe merkt man in der Regel erst ein Zeichen zu spät, dass man die Interpretation schon hätte beenden müssen. Für solche Fälle ist `ungetc` gedacht: Die Funktion bietet die Möglichkeit, ein Zeichen in die Eingabedatei *zurückzuschreiben*! Liest man die Datei anschließend erneut, so erhält man das zurückgeschriebene Zeichen erneut als Eingabe.

Wir sehen uns dafür ein Beispiel an: Eine Eingabezeile soll eine bestimmte Anzahl von Einträgen enthalten, die nacheinander abzuarbeiten sind. Der Benutzer soll nur die ersten

Einträge angeben müssen; fehlende Angaben sollen durch Standardwerte ersetzt werden. Der Einfachheit halber soll angenommen werden, dass die Einträge Zeichen sind und dass der Standardwert das Leerzeichen sein soll. Bei der Lösung müssen wir darauf achten, dass wir das Zeilenende-Zeichen nicht entfernen, solange noch weitere Eingabezeichen folgen. Realisieren lässt sich das durch die Funktion

```
int naechstes_zeichen(void) {
    int c;
    if ((c = getchar ()) == '\n') {
        ungetc (c, stdin);
        c = ' ';
    }
    return c;
}
```

Erwarten wir etwa 40 Zeichen je Zeile, so können wir diese Funktion durch

```
for (i = 0; i < 40; i++) {
    c = naechstes_zeichen();
    ...
}
getchar ();
```

nutzen. Der Aufruf von `getchar` hinter der Schleife sorgt dafür, dass das Zeilenende-Zeichen der nun vollständig abgearbeiteten Eingabezeile aus der Eingabe entfernt wird. Es gibt übrigens eine Reihe von Restriktionen für `ungetc`, auf die hier nicht näher eingegangen werden kann.

